



Lab2 实验报告

一、实现思路

本次实验的目标是在Lab1已经完成词法分析、语法分析和AST构建的基础上，继续实现 SysY 语言的语义分析。语法分析只能判断程序是否符合上下文无关文法，但不能判断程序在语义上是否合法，例如变量是否已经定义、函数调用参数是否匹配、赋值左右两边类型是否一致、数组初始化是否超过容量等。因此，本实验的核心思路是在遍历 AST 的过程中维护符号表，并为表达式节点返回类型信息，从而完成语义规则检查。

我整体分为四个部分实现：类型系统、符号表、类型检查器和数组初始化检查

1.1 类型系统

在类型系统部分，我在模板已有的 `PrimitiveType` 和 `FuncType` 基础上补充了 `ArrayType`。 `ArrayType` 使用 `element_type` 保存数组元素类型，使用 `dims` 保存数组各维度长度。

例如 `int a[2][3]` 可以表示为元素类型为 `int`、维度为 `{2, 3}` 的数组类型。由于类型信息使用 `std::shared_ptr<Type>` 保存，不能直接用 `==` 判断两个类型是否相等(这是直接比较指针是否指向相同的内存)，因此我通过 `compatible` 函数比较类型结构：基本类型比较 `BasicType`，函数类型比较返回值和参数列表，数组类型比较元素类型和维度信息。

1.2 符号表

在符号表部分，我采用了作用域栈的实现方式。符号表内部维护多个作用域，每进入一个新的语句块或函数作用域，就压入一个新的作用域；离开作用域时弹出当前作用域。插入符号时只检查当前作用域是否已有同名符号，从而禁止同一作用域内重复定义；查找符号时则从当前作用域向外层逐层查找，从而支持内部作用域遮蔽外部同名变量。对于符号对象，我保存了符号名、唯一名、类型和作用域编号。其中全局变量和全局函数直接使用原名称作为唯一名，局部变量则根据作用域编号和计数生成唯一名称，方便后续阶段区分不同作用域中的同名变量。

1.3 类型检查器部分

在类型检查器部分，我实现了 `TypeChecker` 类，对 AST 进行递归遍历。`check` 函数根据节

点的实际类型分发到不同的检查函数，例如

`checkFuncDef`、`checkVarDef`、`checkLVal`、`checkFuncCall`、`checkBinaryExp` 等。

- 对于表达式节点，检查函数会返回该表达式的类型
- 对于语句节点，则主要完成语义约束检查。

例如，

- `checkLVal` 负责查找变量是否已经声明，并根据数组下标数量进行类型降维；
- `checkAssignStmt` 检查赋值左侧是否是可赋值对象，并判断左右两侧类型是否兼容；
- `checkFuncCall` 检查被调用对象是否是函数，参数数量是否一致，参数类型是否匹配；
- `checkReturnStmt` 则比较 `return` 表达式类型和当前函数声明返回类型是否一致。

1.4 数组初始化

在数组初始化检查部分，我增加了 `InitList` 节点，用于保存 `{1, 2, {3}}` 这类初始化列表。原本的初始化列表如果只保留第一个元素，就无法判断元素数量是否超出数组容量，因此我修改了 `parser` 中的 `InitVal` 和 `InitVals` 规则，使初始化列表中的所有元素都能被保存到 `AST` 中。

在语义检查阶段，我通过 `check_initializer` 区分普通标量初始化和数组初始化：数组必须使用初始化列表，标量不能使用初始化列表。对于数组初始化，我实现了

`check_array_initializer`，根据数组维度计算总容量，并递归处理嵌套初始化列表。遇到嵌套列表时，根据当前已经填充的元素数量判断它应该对应哪一层子数组，并检查是否存在元素过多或嵌套位置非法的情况。

二、难点与亮点

1. 符号表的作用域管理。变量查找和重复定义检查看起来都和“查找名字”有关，但实际规则不同。普通变量使用时，需要从当前作用域向外层逐层查找；而重复定义检查只应该检查当前作用域，因为内部作用域允许遮蔽外部同名变量。最开始如果不区分这两种查找方式，很容易把合法的变量遮蔽误判为重复定义。最终我通过 `find_symbol(name, true)` 表示只查当前作用域，通过默认查找表示逐层向外查找，从而解决了这个问题。
2. 类型比较。由于类型对象使用 `std::shared_ptr<Type>` 保存，直接比较两个指针只能判断它们是否指向同一个对象，而不能判断它们表示的类型是否相同。例如两个不同对象都可能表示 `int` 类型，它们的指针地址不同，但语义上应当兼容。因此我统一使用 `compatible` 函数判断类型是否匹配。这个设计在数组类型上尤其重要，因为数组类型不仅要比较元素类

型’还要比较维度结构。

3. 数组访问时的类型降维。对于 `int a[2][3][4]`，变量 `a` 的类型是三维数组，`a[0]` 的类型是二维数组，`a[0][1]` 的类型是一维数组，`a[0][1][2]` 的类型才是 `int`。因此在 `checkLVal` 中，我对每一个下标逐层检查：首先判断当前对象是否仍然是数组，再检查下标表达式是否为整数，最后根据下标数量减少数组维度。如果下标数量超过数组维度，就会报出下标对象不是数组的错误。
4. 函数数组参数的匹配。SysY 中函数参数可以写成 `int a[]`、`int a[][2]` 等形式，其中第一维通常省略。为了处理这种情况，我在构造函数参数类型时用 `0` 表示第一维通配，并在函数调用参数检查时对数组参数进行特殊比较：如果形参第一维为 `0`，则允许实参第一维为任意长度，但后续维度必须严格一致。这个细节如果处理不好，会导致本来合法的数组传参被误判为类型不兼容。
5. 数组初始化列表。多维数组初始化不仅要统计元素数量，还要判断嵌套初始化列表应该对应哪一个子数组。例如，当已经填充的元素数可以被某一层子数组容量整除时，嵌套列表才能对应这一层子数组；如果无法对齐，则说明该位置不能使用嵌套列表，应当报错。我通过计算数组从某一维开始的容量，并结合当前已填充元素数量来判断嵌套列表的对应层次。同时，列表初始化结束时未显式填写的部分默认补零，因此递归检查嵌套列表后，需要按照它对应的子数组容量增加已填充元素数，而不是只按列表中显式元素数量增加。

我认为本次实现中的一个亮点是将语义分析拆成了多个相对独立的辅助函数，而不是把所有逻辑堆在一个函数里。例如，类型构造由 `build_var_type` 和 `build_param_type` 完成，初始化检查由 `check_initializer` 和 `check_array_initializer` 完成，数组容量计算由 `array_capacity` 完成。这样每个函数的职责比较清晰，也方便后续调试。

三、心得体会

通过这次实验，我对编译器前端的理解从“能解析代码”进一步推进到了“能判断代码是否合法”。Lab 1 中 AST 主要表示程序的语法结构，而 Lab 2 中符号表和类型系统让 AST 具有了更多语义信息。变量定义时要把类型信息加入符号表，变量使用时要从符号表中找回类型，表达式检查后还要把类型返回给父节点继续判断。这个过程让我更清楚地理解了语义分析在编译流程中的位置。

调试过程中我体会最深的是，语义分析的问题经常不是出现在报错的位置，而是出现在更早的信息传递阶段。例如一开始很多合法的表达式被误判为 `InvalidOperands`，后来发现根本原因是 `checkLVal` 没有正确返回变量类型，导致父节点在检查二元表达式时拿到的不是 `int`。这说明类型检查是一个连续的信息流，只要某个节点返回了错误类型或空类型，后面的赋值、

return 函数调用都会连锁出错。

数组相关的错误也让我印象比较深。尤其是函数数组参数中第一维省略的问题，如果直接按照普通数组类型严格比较维度，就会把合法的函数调用误判为类型错误。后来我用 0 表示形参数组省略的第一维，并在参数兼容判断中专门处理这一情况，才解决了相关测试。这个过程让我意识到，语言规范中的一些细节会直接影响类型系统的设计。

数组初始化检查是本次实验中最复杂的部分。它不像普通赋值那样只需要比较左右类型，而是要理解初始化列表和数组布局之间的对应关系。尤其是嵌套列表出现时，需要根据已经填充的元素个数判断它对应哪一层子数组。实现这一部分时，我先处理一维数组，再扩展到多维数组，最后结合测试用例逐步修正边界情况。这个过程让我感受到，复杂语义规则最好拆成小函数逐步实现，否则很容易把逻辑写乱。

整体来看，本次实验让我更加熟悉了 AST 遍历、作用域管理、类型构造和错误码处理。通过官方测试逐步定位问题也很有帮助：先保证简单变量和函数测试通过，再处理数组下标、函数调用和初始化列表，最后修正少量边界错误。相比直接从大量失败测试中盲目修改，这种分阶段调试的方式效率更高。

四、测试结果

完成实现后，我在仓库根目录下运行了：

```
make clean && make
uv run python3 sp26-tests/test.py lab2 .
```

Running lab2 test...

Running tests 100% 0:00:00

Test Results		
Test File	Result	Time
tests/lab2/add.sy	Accepted	
tests/lab2/arr_defn1.sy	Accepted	
tests/lab2/arr_defn2.sy	Accepted	
tests/lab2/arr_defn3.sy	Accepted	
tests/lab2/arr_defn4.sy	Accepted	
tests/lab2/array1.sy	Accepted	
tests/lab2/array2.sy	Accepted	
tests/lab2/array_init_excess1.sy	Accepted	
tests/lab2/array_init_excess2.sy	Accepted	
tests/lab2/array_init_excess3.sy	Accepted	
tests/lab2/array_init_excess4.sy	Accepted	
tests/lab2/array_init_not_list.sy	Accepted	
tests/lab2/array_lval_error.sy	Accepted	
tests/lab2/assign_mismatch1.sy	Accepted	
tests/lab2/assign_mismatch2.sy	Accepted	
tests/lab2/assign_mismatch3.sy	Accepted	
tests/lab2/cond_with_arr.sy	Accepted	
tests/lab2/exp_init_error.sy	Accepted	
tests/lab2/factorial.sy	Accepted	
tests/lab2/func_defn.sy	Accepted	
tests/lab2/func_name.sy	Accepted	
tests/lab2/funcparam.sy	Accepted	
tests/lab2/if_complex_expr.sy	Accepted	
tests/lab2/if_test1.sy	Accepted	
tests/lab2/if_test2.sy	Accepted	
tests/lab2/invalid_call.sy	Accepted	
tests/lab2/main.sy	Accepted	
tests/lab2/mul.sy	Accepted	
tests/lab2/op_priority.sy	Accepted	
tests/lab2/redef_fparam.sy	Accepted	
tests/lab2/redef_fun.sy	Accepted	
tests/lab2/redef_var.sy	Accepted	
tests/lab2/rem.sy	Accepted	
tests/lab2/return_mismatch1.sy	Accepted	
tests/lab2/return_mismatch2.sy	Accepted	
tests/lab2/return_mismatch3.sy	Accepted	
tests/lab2/scope.sy	Accepted	
tests/lab2/stmt_expr.sy	Accepted	
tests/lab2/subscript_error1.sy	Accepted	
tests/lab2/subscript_error2.sy	Accepted	
tests/lab2/subscript_error3.sy	Accepted	
tests/lab2/type_incompatible.sy	Accepted	
tests/lab2/undef_func.sy	Accepted	
tests/lab2/undef_variable1.sy	Accepted	
tests/lab2/undef_variable2.sy	Accepted	
tests/lab2/unmatch_binop.sy	Accepted	
tests/lab2/unmatch_call_array1.sy	Accepted	
tests/lab2/unmatch_call_array2.sy	Accepted	
tests/lab2/unmatch_call_num.sy	Accepted	
tests/lab2/unmatch_call_type.sy	Accepted	
tests/lab2/unmatch_unop.sy	Accepted	
tests/lab2/while_cond_error.sy	Accepted	
tests/lab2/while_if.sy	Accepted	
tests/lab2/while_test.sy	Accepted	

All tests passed!

五、借鉴声明

本实验未直接复用他人完整代码或公开参考实现。实现过程中主要依据课程实验文档、模板代码注释、官方测试反馈以及本地调试结果完成。若有参考课程实验文档中的规则描述或模板接口，均属于实验提供材料范围。

六、AI工具使用记录

本实验过程中使用了 ChatGPT 作为问答类 AI 工具，主要用于理解实验文档、解释 C++ 语法点、分析编译报错和测试错误、梳理符号表与类型检查器的实现思路，以及辅助定位数组初始化和函数数组参数匹配相关问题。代码修改、编译运行和测试均在本地环境中完成，并根据项目实际结构进行了人工调整。

lab2.md
文件

我们现在有这样一个实验，希望你能带我完成，先大致介绍一下整个实验



我先按“你要做什么、代码要改哪里、检查哪些错误、最后怎么测试提交”这条线帮你把 Lab 2 梳理清楚，后面我们可以再逐个文件推进实现。

Thought for 14s >

这个 Lab 2 的核心可以一句话概括：

在 Lab 1 已经能生成 AST 的基础上，继续遍历 AST，完成“语义分析”：检查变量、函数、类型、数组初始化等是否真的合法。

lab2

1. 这个实验到底在做什么？

Lab 1 解决的是：

C

```
int a = 1 + 2;
```



这段代码在语法上能不能被识别，能不能构建出语法树。

Lab 2 要解决的是：

C

```
int a[5] = {1, 2, 3, 4, 5};  
int b = a;
```



这段代码虽然语法没错，但是语义不合法，因为 **a** 是数组，**b** 是整数，类型不匹配。

所以 Lab 2 的目标就是：

判断一个 SysY 程序不仅“写得像代码”，而且“语义上真的正确”。

2. 你主要要实现三大块

第一块：类型系统 `type.hpp / type.cpp`

实验已经给了基础类型：

```
</> C++  
  
int  
void  
function
```



你还需要重点实现：

```
</> C++  
  
ArrayType
```



也就是数组类型。

比如：

```
</> C  
  
int a[2][3];
```



它的类型可以理解为：

```
array[2][3] of int
```



后面检查赋值、初始化、函数参数、数组下标时，都要依赖这个类型信息。

注意：类型比较不能直接用 `==`，而是要用：

</> C++

typedef __comptime_t type;

第二块：符号表 `symbol_table.hpp` / `symbol_table.cpp`

符号表的作用是记录：

变量名 / 函数名 -> 它的类型、作用域、唯一名字

例如：

</> C

```
int x = 1;
{
    int x = 2;
}
```

这两个 `x` 名字一样，但不是同一个变量。

所以符号表要支持：

</> C++

```
add_symbol(name, type);    // 添加符号
find_symbol(name);         // 查找符号
find_symbol(name, true);   // 只在当前作用域查找
enter_scope();             // 进入新作用域
exit_scope();              // 退出作用域
```

你要保证：

</> C

```
int x;
int x;    // 同一作用域重复定义，报错 11
```

`</> C++`



你要保证：

`</> C`



```
int x;  
int x;    // 同一作用域重复定义，报错 11
```

但允许：

`</> C`



```
int x;  
{  
    int x;    // 内层遮蔽外层，合法  
}
```

第三块：类型检查器 `type_checker.hpp` / `type_checker.cpp`

这是整个实验的核心。

它会遍历 AST，每遇到一种节点，就调用对应的 `check` 函数：

`</> C++`



```
checkCompUnit  
checkFuncDef  
checkVarDef  
checkLVal  
checkBinaryExp  
checkFuncCall  
checkReturnStmt  
...
```

每个 `check` 函数主要做两件事：



第一，检查这个节点是否合法。

第二，返回这个节点的类型。

比如：

```
</> C
```

```
a + b
```



你要检查：

a 是不是 int

b 是不是 int



如果有一边不是 int，就报：

5 InvalidOperands



再比如：

```
</> C
```

```
int x = f(1, 2);
```



你要检查：

f 是不是函数

参数个数对不对

参数类型对不对

返回值能不能赋给 x



3. 需要检查哪些语义错误？

可以按这几类理解。



1) 名字和作用域错误

例如：

例如：

```
</> C
int main() {
    x = 1;
}
```

x 没定义，要报：

10 Undeclared

例如：

```
</> C
int a;
int a;
```

同一作用域重复定义，要报：

11 Redefinition

2) 赋值和初始化错误

例如：

```
</> C
int a[5];
int b = a;
```

数组不能直接赋给整数，类型不兼容，要报：



4 IncompatibleConversion

例如：

```
</> C
int a[2];
a = 1;
```



数组整体不能作为赋值左边，要报：

2 NotAssignable



3) 运算符错误

例如：

```
</> C
void f() {}
int x = 1 + f();
```



f() 返回 void，不能参与加法，要报：

5 InvalidOperands



所有算术、比较、逻辑运算的操作数基本都应该是 int。

4) 函数调用错误

例如：

```
</> C
int f(int x) {
    return x;
}
```



参数数量不匹配，要报：

13 ArityMismatch



例如：

`</> C`



```
int a;  
a();
```

a 不是函数，不能调用，要报：

9 NotCallable



5) 数组下标错误

例如：

`</> C`



```
int a;  
a[0];
```

a 不是数组，要报：

6 NotSubscriptable



例如：

`</> C`



```
int a[10];  
a[void_func()];
```



下标不是整数，要报：

6) 返回值错误

例如：

```
</> C
int f() {
    return;
}
```



`int` 函数不能空返回，要报：

7 ReturnMismatch



例如：

```
</> C
void f() {
    return 1;
}
```



`void` 函数不能返回整数，也要报：

7 ReturnMismatch



7) 数组初始化错误

这是本实验最难的部分。



例如：

 C



```
int a[2][2] = {1, 2, 3, 4};
```

合法，相当于：

```
a[0][0] = 1  
a[0][1] = 2  
a[1][0] = 3  
a[1][1] = 4
```



但是：

 C



```
int a[2][2] = {1, 2, 3, 4, 5};
```

多了一个 5，要报：

3 ExcessInitializers



还有这种：

 C



```
int a[2] = 1;
```

数组初始化必须用初始化列表，不能直接用标量，要报：

12 InvalidInitializer



数组初始化建议最后做，因为它涉及多维数组、嵌套 {}、自动补 0、超出容量判断，逻辑最复杂。



4. 推荐完成顺序

我建议你不要一上来就写 `type_checker.cpp`，而是按这个顺序来：

第一步：先切分支并合并模板

`</>` Bash



```
git checkout -b lab2 lab1
git merge upstream/template/lab2
git push -u origin lab2
```

第二步：实现类型系统

重点补：

`</>` C++



```
ArrayType
ArrayType::compatible
ArrayType::to_string
```

目标是能正确表示：

`</>` C



```
int a[2][3];
```

这种数组类型。

第三步：实现符号表

重点补：

`</>` C++



```
add_symbol
find_symbol
```

 C++ 

```
enter_scope  
exit_scope
```

先把作用域管理跑通。

这一部分是后面所有类型检查的基础。

第四步：实现基础 TypeChecker

先支持最简单程序：

 C 

```
int main() {  
    return 0;  
}
```

然后逐步支持：

变量定义
变量引用
赋值语句
表达式运算
return
if / while 条件
函数定义
函数调用
数组下标
数组初始化 

第五步：处理错误码



实验不是只看你有没有报错，而是看你有没有返回正确的 exit code。

```
0 OK
1 SyntaxError
2 NotAssignable
3 ExcessInitializers
4 IncompatibleConversion
5 InvalidOperands
6 NotSubscriptable
7 ReturnMismatch
8 NotIntegerSubscript
9 NotCallable
10 Undeclared
11 Redefinition
12 InvalidInitializer
13 ArityMismatch
```

建议统一写一个语义错误处理函数，比如：

```
</> C++
void semantic_error(ErrorCode code, AST::NodePtr node, std::string message);
```

然后里面打印错误并 `exit(code)`。

5. 怎么测试？

在仓库根目录下运行：

```
</> Bash
python3 sp26-tests/test.py lab2 .
```

你的编译器必须支持：

```
</> Bash
./compiler <input_file>
```

你的编译器必须支持。

`</> Bash`

```
./compiler <input_file>
```

合法程序返回：

0

非法程序返回对应错误码。

6. 这个实验最重要的理解

Lab 2 其实就是让编译器从：

“这段代码能不能被语法分析器读懂”

升级到：

“这段代码在语言规则上到底合不合法”

实现过程中，你可以把整体流程想成：

AST

↓

TypeChecker 遍历 AST

↓

遇到定义：加入符号表

遇到使用：查符号表

遇到表达式：返回类型

遇到赋值/调用/返回/数组：检查类型是否符合规则

↓

无错误：返回 0

有错误：返回指定错误码

↓

```
(base) 🐚 base 📄 yangsmac@YangsdeMacBook-Air 📄  
~/Desktop/compiler/cp-3230104703 📄 📄 lab2 📄 git  
merge upstream/template/lab2  
Auto-merging src/ast/tree.hpp  
CONFLICT (content): Merge conflict in src/ast/tree.hpp  
Auto-merging src/common.hpp  
Automatic merge failed; fix conflicts and then commit the  
result.这种问题如何解决
```

Thought for 8s >

这是 Git 合并冲突，不是代码编译错误。意思是：

你的 lab2 分支里改过 src/ast/tree.hpp，而 upstream/template/lab2 里也改过这个文件，Git 不知道应该保留哪一版，所以需要手动处理。

实验指导里也说了：合并模板代码时如果出现冲突，需要手动编辑冲突文件，然后 git add 和 git commit。📄 lab2

1. 先查看当前冲突状态

在仓库根目录运行：

```
</> Bash  
git status
```

你大概率会看到类似：

```
both modified: src/ast/tree.hpp
```

说明现在只有 tree.hpp 需要你手动解决。

2. 打开冲突文件

可以用 VS Code:

```
</> Bash  
code src/ast/tree.hpp
```



或者直接用编辑器打开:

```
</> Bash  
open src/ast/tree.hpp
```



你会看到这种冲突标记:

```
</> C++  
  
<<<<<<< HEAD  
// 你当前 lab2 分支里的内容  
=====  
// upstream/template/lab2 里的内容  
>>>>>>> upstream/template/lab2
```



这里:

```
HEAD = 你自己当前分支里的版本  
incoming / upstream/template/lab2 = 模板分支带来的版本
```



3. 处理原则: 不要简单全选某一边

这个实验里, `tree.hpp` 很关键。

Lab 1 里你可能已经改过 AST 节点。



Lab 2 模板又会往 AST 节点里加一些语义分析需要的字段, 比如:

 C++



```
SymbolPtr symbol;
```

所以最稳妥的处理方式是：

保留你 Lab 1 已经能正常语法分析的 AST 结构，同时把 Lab 2 新增的语义字段合进去。

比如实验里提到，Lab 2 会在某些节点中加入符号表信息：

 C++



```
class VarDef : public Node {  
public:  
    std::string ident;  
    SymbolPtr symbol;  
    ...  
};
```

类似这种字段一般要保留，因为后面的语义分析和 IR 生成可能会用到。

4. VS Code 里怎么选？

如果你用 VS Code，会出现按钮：

Accept Current Change
Accept Incoming Change
Accept Both Changes
Compare Changes



含义是：

Accept Current Change = 保留你自己的版本
Accept Incoming Change = 保留模板版本
Accept Both Changes = 两边都保留



这个文件建议你不要无脑选一个。

这个文件建议你不要无脑选一个。

一般可以先选：

Accept Both Changes



然后手动整理重复代码，尤其注意：

1. 不要出现重复的 class 定义。
2. 不要删除你 Lab 1 里 parser 正在使用的 AST 节点。
3. 要保留 Lab 2 新增的 `SymbolPtr symbol`。
4. 如果模板新增了 include，也要保留。

例如可能需要保留类似：

`</> C++`



```
#include "../semantic/symbol_table.hpp"
```

或者前向声明相关内容。

5. 解决完后删除冲突标记

最终文件里不能再有这些东西：

`</> C++`



```
<<<<<<< HEAD
=====
>>>>>>> upstream/template/lab2
```

可以用命令检查：

`</> Bash`



```
grep -n "<<<<<<<\|=====\|>>>>>>>" src/ast/tree.hpp
```

如果没有输出，说明冲突标记已经清理干净。

6. 标记冲突已解决

编辑完后运行：

```
</> Bash
```

```
git add src/ast/tree.hpp
```

然后提交合并结果：

```
</> Bash
```

```
git commit
```

如果 Git 自动弹出编辑器写 merge commit 信息，不用改，直接保存退出即可。

如果你想自己写信息：

```
</> Bash
```

```
git commit -m "Merge lab2 template and resolve AST conflicts"
```

最后推送：

```
</> Bash
```

```
git push origin lab2
```

7. 如果你觉得改乱了，可以放弃这次合并

如果你还没 `git add` / `git commit`，可以回到合并前：

```
</> Bash
```

```
git merge --abort
```



然后重新来：

然后重新来：

 Bash



```
git merge upstream/template/lab2
```

8. 最推荐你的实际操作顺序

直接按这个来：

 Bash



```
git status  
code src/ast/tree.hpp
```

在 VS Code 中手动解决冲突，重点保留：

你 Lab 1 的 AST 节点结构

+

Lab 2 模板新增的 SymbolPtr symbol 等语义字段



然后：

 Bash



```
grep -n "<<<<<<\|=====\\|>>>>>>" src/ast/tree.hpp  
git add src/ast/tree.hpp  
git commit -m "Merge lab2 template"  
git push origin lab2
```

如果你愿意，可以把 `src/ast/tree.hpp` 里冲突的那一段贴给我，我可以帮你判断应该保留哪一部分。

     ...  来源



```
(base) x  base yangsmac@YangsdeMacBook-Air ~/Desktop/compiler/cp-3230104703 lab2 ±  
>M< git status
```

On branch lab2

Your branch is up to date with 'origin/lab2'.

You have unmerged paths.

(fix conflicts and run "git commit")

(use "git merge --abort" to abort the merge)

Changes to be committed:

modified: src/common.hpp

modified: src/main.cpp

new file: src/semantic/symbol_table.cpp

new file: src/semantic/symbol_table.hpp

new file: src/semantic/type.cpp

new file: src/semantic/type.hpp

new file: src/semantic/type_checker.cpp

new file: src/semantic/type_checker.hpp

Unmerged paths:

(use "git add <file>..." to mark resolution)

both modified: src/ast/tree.hpp

Thought for 7s >

你这个状态说明：模板代码大部分已经合并成功了，只有 `src/ast/tree.hpp` 还在冲突中。

实验指导里说的也是这种情况：冲突文件需要手动编辑，解决后再 `git add` 和 `git commit`。

lab2



你现在不要做什么？

先不要直接：

```
</> Bash
```



```
git commit
```

因为还有这个文件没解决：

```
both modified: src/ast/tree.hpp
```



你现在要做什么？

第一步：打开冲突文件

用 VS Code：

```
</> Bash
```



```
code src/ast/tree.hpp
```

或者普通方式打开：

```
</> Bash
```



```
open src/ast/tree.hpp
```

第二步：找到冲突标记

在文件里搜索：

```
</> C++
```



```
<<<<<<<<
```



你会看到类似：

```
</> C++  
  
<<<<<<< HEAD  
// 你 lab1 / 当前分支中的内容  
=====  
// upstream/template/lab2 中的内容  
>>>>>>> upstream/template/lab2
```

含义是：

```
HEAD = 你当前 lab2 分支的代码  
incoming = lab2 模板代码
```

关键处理原则

`tree.hpp` 不能简单选一边。

你需要做到：

```
保留你 Lab 1 里已有的 AST 节点  
+  
合并 Lab 2 新增的语义分析字段
```

尤其注意 Lab 2 可能新增了这些东西：

```
</> C++  
  
SymbolPtr symbol;
```

比如这些节点里可能会多出 `symbol`：

```
</> C++  
  
VarDef
```

 C++



```
Level  
FuncDef  
FuncCall
```

这些字段后面 `TypeChecker` 会用到，最好保留。

VS Code 里怎么选？

如果 VS Code 顶部有这些按钮：

```
Accept Current Change  
Accept Incoming Change  
Accept Both Changes  
Compare Changes
```



建议你优先点：

Compare Changes



看清楚两边差异。

如果两边都重要，可以点：

Accept Both Changes



然后手动整理重复代码。

最终目标是：

1. 不要有重复 class。
2. 不要删掉你 Lab 1 parser 依赖的 AST 节点。
3. 保留 Lab 2 新增的 `SymbolPtr symbol` 
4. 保留 Lab 2 需要的新头文件或前向声明。
5. 文件里不能再有 `<<<<<<<`、`=====`、`>>>>>>>`。

5. 文件里不能再有 <<<<<<<、=====、>>>>>>>。

第三步：检查冲突标记是否清除

保存文件后，在终端运行：

```
</> Bash
grep -n "<<<<<<<\\|=====\\|>>>>>>>" src/ast/tree.hpp
```

如果没有任何输出，说明冲突标记已经清理干净。

第四步：标记冲突已解决

```
</> Bash
git add src/ast/tree.hpp
```

然后再看状态：

```
</> Bash
git status
```

如果正常，会看到类似：

```
All conflicts fixed but you are still merging.
```

或者所有文件都在 `Changes to be committed` 下面。

第五步：提交合并



```
</> Bash
```

第五步：提交合并

 Bash 

```
git commit -m "Merge lab2 template"
```

然后推送：

 Bash 

```
git push origin lab2
```

你现在最短操作流程

直接按这个来：

 Bash 

```
code src/ast/tree.hpp
```

手动解决冲突并保存，然后：

 Bash 

```
grep -n "<<<<<<\|=====\|>>>>>>" src/ast/tree.hpp
git add src/ast/tree.hpp
git status
git commit -m "Merge lab2 template"
git push origin lab2
```

如果你把 `src/ast/tree.hpp` 里出现 `<<<<<<` 的那一段贴出来，我可以直接帮你判断这一段应该怎么合并。

     ...  来源





粘贴的文本 (1).txt
文档

这是什么意思，是否有问题

Thought for 5s >

这个输出没有严重问题，因为最后一行是：

```
</> Bash
```



```
g++ -o compiler ...
```

说明已经成功生成了可执行文件：

```
</> Bash
```



```
./compiler
```

也就是说：编译通过了，没有 error，只有 warning。

粘贴的文本 (1)

这些 warning 是什么意思？

这些 warning 基本都是模板代码里故意留下的：

```
</> C++
```



```
#warning Not implemented: ...
```

意思是：

这里还没有实现，提醒你后面要补代码。



比如输出里有：

```
warning: Not implemented: SymbolTable
warning: Not implemented: SymbolTable::add_symbol
warning: Not implemented: SymbolTable::find_symbol
warning: Not implemented: SymbolTable::enter_scope
warning: Not implemented: SymbolTable::exit_scope
```



这说明：符号表还没实现。这是我们下一步要做的内容。

还有：

```
warning: Not implemented: TypeChecker::checkFuncDef
warning: Not implemented: TypeChecker::checkVarDef
warning: Not implemented: TypeChecker::checkLVal
warning: Not implemented: TypeChecker::checkFuncCall
```



这说明：类型检查器还没实现，这个是后面的大头。

目前最关键的结论

你的 `type.cpp` / `type.hpp` 这部分目前至少已经做到：

```
可以被编译器接受
没有语法错误
没有链接错误
```



尤其这一行：

```
g++ -std=c++17 -g -Wall -MMD -Isrc -c -o src/semantic/type.o src/semantic/type
```



没有报错，说明 `ArrayType` 这部分目前编译通过了。

粘贴的文本 (1)

现在是否需要处理这些 warning?



分情况：

万得/元·

现在阶段：不用急着全部消除。

但是最后提交前：必须把核心的 Not implemented 都实现掉。

特别是这些必须后续解决：

```
SymbolTable::add_symbol  
SymbolTable::find_symbol  
SymbolTable::enter_scope  
SymbolTable::exit_scope  
TypeChecker::checkXXX
```



否则即使 `make` 能过，测试也过不了。

下一步该做什么？

现在第一部分 `Type / ArrayType` 已经基本完成。

下一步进入第二部分：

```
src/semantic/symbol_table.hpp  
src/semantic/symbol_table.cpp
```



也就是实现：

```
</> C++  
  
add_symbol()  
find_symbol()  
enter_scope()  
exit_scope()
```



这一部分做完之后，很多 `SymbolTable` 相关 warning 就会消失，后面才能正式写 `TypeChecker`。



Thought for a second >

`std::move()` 的作用是：

把一个对象“标记”为可以被移动资源”，从而优先调用移动构造 / 移动赋值，而不是拷贝。

它本身不真的搬东西，只是做一个类型转换。

1. 普通拷贝是什么？

比如：

`</> C++`

```
std::string a = "hello";  
std::string b = a;
```

这里是拷贝：

a 里面有一份 "hello"
b 里面也复制出一份 "hello"

拷贝可能比较耗时，尤其是字符串、vector、map 这种内部有大量数据的对象。

2. `std::move` 是什么？

`</> C++`

```
std::string a = "hello";  
std::string b = std::move(a);
```

 C++ 

```
std::string a = "hello";  
std::string b = std::move(a);
```

这里表示：

我之后不再需要 **a** 原来的内容了，你可以把 **a** 里面的资源“转移”给 **b**。

可以理解为：

不是复制一份 hello
而是把 a 管理的那块资源交给 b 

所以效率更高。

3. 在你代码里的作用

你写过类似：

 C++ 

```
Symbol(std::string name, TypePtr type, int scope_id = 0)  
    : name(std::move(name)), type(std::move(type)), scope_id(scope_id) {}
```

这里的意思是：

构造函数参数 name 已经是一个临时传进来的字符串
我们不需要再复制一份
可以直接把它移动到成员变量 this->name 里 

相比：

 C++  

```
: name(name), type(type)
```

`std::move(name)` 更高效，尤其当 `name` 很长时。

4. 一个具体例子

`</> C++`



```
std::vector<int> a = {1, 2, 3, 4, 5};  
std::vector<int> b = std::move(a);
```

移动后：

b 拿到了原来 a 的数据
a 仍然是合法对象，但内容不保证还在



也就是说，移动后不能假设：

`</> C++`



```
a[0]
```

还能正常得到 1。

但你可以继续给 `a` 重新赋值：

`</> C++`



```
a = {10, 20};
```

5. 一个常见误区

`std::move(a)` 并不会立刻清空 `a`。

它只是告诉编译器：



a 可以被当成“将亡值”使用



a 可以被当成“将亡值”使用
可以优先调用移动构造函数



真正发生资源转移，是在这里：

`</> C++`



```
std::string b = std::move(a);
```

也就是调用移动构造时。

6. 对你现在实验来说

在这些地方用 `std::move` 是合理的：

`</> C++`



```
Symbol(std::string name, TypePtr type, int scope_id = 0)  
    : name(std::move(name)), type(std::move(type)), scope_id(scope_id) {}
```

`</> C++`

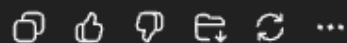


```
ArrayType(TypePtr element_type, std::vector<int> dims)  
    : element_type(std::move(element_type)), dims(std::move(dims)) {}
```

因为这些参数传进来后，通常就不会再用了，直接移动到成员变量里更高效。

一句话总结：

`std::move(x)` = 告诉 C++：这个 x 我可以不要原来的内容了，你可以把它的资源转移走。



Thought for a second >

`back()` 和 `end()` 都是 C++ 容器常用方法，但作用不一样。

1. `back()` 是什么？

`back()` 表示：取容器的最后一个元素。

比如：

```
</> C++  
  
std::vector<int> a = {10, 20, 30};  
  
std::cout << a.back(); // 输出 30
```

在你的符号表代码里：

```
</> C++  
  
auto& current_scope = scopes.back();
```

意思是：

取 `scopes` 这个作用域栈的最后一层，也就是“当前作用域”

因为我们用 `vector` 模拟作用域栈：

<code>scopes[0]</code>	全局作用域
<code>scopes[1]</code>	函数作用域
<code>scopes[2]</code>	当前 block 作用域



所以：

所以：

```
</> C++  
  
scopes.back()
```

就是当前最内层作用域。

2. `end()` 是什么？

`end()` 表示：容器最后一个元素的后一个位置。

注意：它不是最后一个元素。

例如：

```
</> C++  
  
std::vector<int> a = {10, 20, 30};
```

可以想象成：

```
begin()                end()  
  ↓                    ↓  
[10] [20] [30] [不存在的位置]
```

所以 `end()` 常用于判断“有没有找到”。

3. `find(...) != end()` 是什么意思？

比如：

```
</> C++  
  
std::unordered_map<std::string, int> mp;
```

 C++



```
mp["x"] = 1;
```

查找：

 C++



```
auto it = mp.find("x");
```

如果找到了，`it` 指向 `"x"` 这一项。

如果没找到，`it == mp.end()`。

所以：

 C++



```
if (mp.find("x") != mp.end()) {  
    // 找到了 x  
}
```

意思是：

如果 `find` 的结果不是 `end`，说明真的找到了



反过来：

 C++



```
if (mp.find("x") == mp.end()) {  
    // 没找到 x  
}
```

意思是：



如果 `find` 的结果等于 `end`，说明没有找到



4. 放到你的代码里看

</> C++



```
auto& current_scope = scopes.back();

if (current_scope.find(name) != current_scope.end()) {
    return nullptr;
}
```

逐句解释：

</> C++



```
auto& current_scope = scopes.back();
```

取出当前作用域。

</> C++



```
current_scope.find(name)
```

在当前作用域中查找变量名 `name`。

</> C++



```
current_scope.end()
```

表示“没找到时返回的位置”。

所以整个判断：

</> C++



```
if (current_scope.find(name) != current_scope.end())
```



意思是：

意思是：

如果当前作用域里已经存在 `name` 这个符号，说明重复定义，返回 `nullptr`。

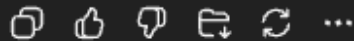
例如：

```
</> C  
  
int x;  
int x;
```

第二次插入 `x` 时，`find("x") != end()` 成立，所以返回 `nullptr`。

5. 一句话记忆

`back()`：取最后一个元素
`end()`：表示“末尾之后的位置”，常用来判断 `find` 是否失败
`find(x) != end()`：找到了 `x`
`find(x) == end()`：没找到 `x`



```
if(in_current_scope) {  
    const auto& current_scope = scopes.back();  
    auto it = current_scope.find(name);  
  
    if(it != current_scope.end()) {  
        return it->second;  
    }  
  
    return nullptr;  
}这是什么意思
```

Thought for a second >

这段代码的意思是：

如果 `in_current_scope == true`，那就只在当前作用域里查找这个符号，不去外层作用域找。

逐行解释：

`</> C++`



```
if (in_current_scope) {
```

如果参数 `in_current_scope` 为 `true`，进入这个分支。

比如后面你检查重复定义时会这样用：

`</> C++`



```
find_symbol("x", true)
```

意思是：只检查当前作用域有没有 `x`。



 C++



```
const auto& current_scope = scopes.back();
```

`scopes` 是一个作用域栈。

比如：

```
scopes[0] = 全局作用域  
scopes[1] = main 函数作用域  
scopes[2] = 当前 if / while / block 作用域
```



`scopes.back()` 取最后一个，也就是当前最内层作用域。

`const auto&` 的意思是：

用引用拿到当前作用域，不拷贝它；
`const` 表示这里不会修改它。



 C++



```
auto it = current_scope.find(name);
```

在当前作用域里查找名字为 `name` 的符号。

假设：

 C++



```
name = "x";
```

那这句就是：



 C++



`</> C++`



```
current_scope.find("x");
```

`</> C++`



```
if (it != current_scope.end()) {  
    return it->second;  
}
```

如果 `it != current_scope.end()`，说明找到了。

因为 `current_scope` 是一个 `map`，大概长这样：

`</> C++`



```
"x" -> SymbolPtr  
"y" -> SymbolPtr
```

所以：

`</> C++`



```
it->first    // 符号名, 比如 "x"  
it->second   // 对应的 SymbolPtr
```

因此：

`</> C++`



```
return it->second;
```

就是返回找到的符号对象。



`</> C++`



```
return nullptr;
```

如果没找到，就返回空指针，表示当前作用域没有这个符号。

举个例子：

`</> C`



```
int x;

int main() {
    int y;
}
```

当前在 `main` 里面，当前作用域只有：

`y`



如果调用：

`</> C++`



```
find_symbol("y", true)
```

会找到 `y`。

如果调用：

`</> C++`



```
find_symbol("x", true)
```



找不到，因为 `x` 在外层全局作用域，而这个分支只查当前作用域。

所以这段代码主要用于判断：

```
</> C   
  
int y;  
int y; // 当前作用域重复定义
```

一句话总结：

```
</> C++   
  
if (in_current_scope) { ... }
```

这一段就是：只查当前作用域，找到了就返回符号，没找到就返回 `nullptr`。

     ...

由于我们都使用了 `std::shared_ptr<Type>` 来储存类型信息，所以在比较两个类型是否相等时，需要使用 `compatible` 函数而不是直接使用 `==` 或 `!=` 你能解释一下这段话的因果关系吗

Thought for a few seconds >

这句话的因果关系是：

因为我们保存类型时用的是 `std::shared_ptr<Type>`，所以变量里存的不是“类型本身”，而是“指向类型对象的指针”。
因此，直接用 `==` / `!=` 比较，比较的是两个指针是不是指向同一个对象，而不是比较两个类型在语义上是否相同。

举个例子：

 C++



```
auto t1 = PrimitiveType::create(BasicType::Int);  
auto t2 = PrimitiveType::create(BasicType::Int);
```

从语义上看：

t1 是 int
t2 也是 int



所以它们应该是同一种类型。

但是：

 C++



```
t1 == t2
```

结果可能是 `false`，因为 `t1` 和 `t2` 是两个不同的 `shared_ptr`，分别指向两个不同的 `PrimitiveType` 对象。

可以想象成：

t1 -> 一个 `int` 类型对象，地址 `0x1000`
t2 -> 另一个 `int` 类型对象，地址 `0x2000`

它们内容都表示 `int`，但地址不同。

所以不能这样判断：

```
</> C++  
  
if (left_type == right_type) {  
    // 类型相同  
}
```

因为这判断的是：

`left_type` 和 `right_type` 是不是同一个对象

而我们真正想判断的是：

`left_type` 和 `right_type` 表示的类型结构是否相同

所以要写：

```
</> C++  
  
if (left_type->compatible(right_type)) {  
    // 类型兼容  
}
```

比如 `PrimitiveType::compatible` 里面一般会这样写：

```
</> C++  
  
bool PrimitiveType::compatible(const TypePtr& other) const {  
    auto other_type = std::dynamic_pointer_cast<PrimitiveType>(other);  
    return other_type && basic_type == other_type->basic_type;  
}
```

这才是在比较：

对方是不是 `PrimitiveType`
并且 `basic_type` 是否相同

也就是：

`int` 和 `int` 兼容
`int` 和 `void` 不兼容

再看数组类型：

```
</> C++  
  
int a[2][3];  
int b[2][3];
```

它们可能分别对应两个不同的 `ArrayType` 对象：

```
</> C++  
  
auto arr1 = ArrayType::create(PrimitiveType::Int, {2, 3});  
auto arr2 = ArrayType::create(PrimitiveType::Int, {2, 3});
```

如果用：

`</> C++`



比较的是地址，可能是 `false`。

但如果用：

`</> C++`



```
arr1->compatible(arr2)
```

比较的是：

元素类型是不是一样
数组维度是不是一样



所以应该返回 `true`。

一句话总结：

`shared_ptr` 的 `==` 比较的是“是不是同一个对象”；
`compatible` 比较的是“两个对象表示的类型是否相同”。



所以在语义分析里，判断类型是否匹配时，要用：

`</> C++`



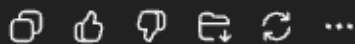
```
type1->compatible(type2)
```

而不是：

`</> C++`



```
type1 == type2
```



```
src/semantic/type_checker.cpp:10:19: error: out-of-line
definition of 'error' does not match any declaration in
'TypeChecker'
```

```
10 | void TypeChecker::error(int code, AST::NodePtr
node,
```

```
|           ^~~~~~
src/semantic/type_checker.hpp:22:49: note: type of 3rd
parameter of member declaration does not match
definition ('const std::string' (aka 'const
basic_string<char>') vs 'const std::string &' (aka 'const
basic_string<char> &'))
```

```
22 | void error(int code, AST::NodePtr node, const
std::string message);
```

```
|           ^
src/semantic/type_checker.cpp:45:22: error: out-of-line
definition of 'build_var_type' does not match any
declaration in 'TypeChecker'
```

```
45 | TypePtr
TypeChecker::build_var_type(AST::FuncParamPtr param)
{上述报错是什么原因造成的
```

Thought for a couple of seconds >

这两个报错都是函数声明和函数实现不一致导致的，不是逻辑错误。

报错 1: `error` 的声明和实现不一致

报错信息:

```
out-of-line definition of 'error' does not match any declaration in 'TypeCheck
```

你的 `.hpp` 里声明是:

你的 `.hpp` 里声明是：

`</> C++`

```
void error(int code, AST::NodePtr node, const std::string message);
```

但是 `.cpp` 里实现是：

`</> C++`

```
void TypeChecker::error(int code, AST::NodePtr node,  
                        const std::string& message) {
```

区别在第三个参数：

`</> C++`

```
const std::string message
```

和：

`</> C++`

```
const std::string& message
```

这两个在 C++ 里不是同一个函数签名，所以编译器认为：

你在 `.cpp` 里实现了一个我没在 `.hpp` 里声明过的函数。

修改方法

把 `type_checker.hpp` 里的声明改成：

`</> C++`

```
void error(int code, AST::NodePtr node, const std::string& message);
```

也就是加上 `&`。

为什么要用 `const std::string&`?

`</> C++`



```
const std::string& message
```

表示：

只读引用，不拷贝字符串



这样比直接传 `std::string` 更高效。

报错 2: `build_var_type` 的声明和实现不一致

报错信息：

out-of-line definition of 'build_var_type' does not match any declaration in '



你的 `.cpp` 里现在可能写成了：

`</> C++`



```
TypePtr TypeChecker::build_var_type(AST::FuncParamPtr param) {
```

但这明显不对。

因为我们前面设计的是两个函数：

`</> C++`



```
TypePtr build_var_type(BasicType btype, const std::vector<int>& dims);  
TypePtr build_param_type(AST::FuncParamPtr param);
```


也就是说：

- `build_var_type`：处理普通变量定义，比如 `int a[2][3]`
- `build_param_type`：处理函数参数，比如 `int a[][3]`

你现在应该是把函数名写错了。

正确写法应该是这样

`type_checker.hpp` 里声明：

`</> C++`



```
TypePtr build_var_type(BasicType btype, const std::vector<int>& dims);  
TypePtr build_param_type(AST::FuncParamPtr param);
```

`type_checker.cpp` 里实现：

`</> C++`



```
TypePtr TypeChecker::build_var_type(BasicType btype,  
                                     const std::vector<int>& dims) {  
    TypePtr base = PrimitiveType::create(btype);  
  
    if (dims.empty()) {  
        return base;  
    }  
  
    return ArrayType::create(base, dims);  
}
```

这段是给变量定义用的。



比如：

 C

```
int x;
```

得到 `int`。

 C

```
int a[2][3];
```

得到 `array[2][3] of int`。

然后再单独写函数参数版本：

 C++

```
TypePtr TypeChecker::build_param_type(AST::FuncParamPtr param) {
    TypePtr base = PrimitiveType::create(param->btype);

    if (!param->is_array) {
        return base;
    }

    std::vector<int> dims;
    dims.push_back(0);

    if (param->array_second_dim > 0) {
        dims.push_back(param->array_second_dim);
    }

    if (param->array_third_dim > 0) {
        dims.push_back(param->array_third_dim);
    }

    return ArrayType::create(base, dims);
}
```



这段是给函数参数用的。

比如：

`</> C`

```
int f(int x)
```



参数类型是 `int`。

`</> C`

```
int f(int a[][3])
```



参数类型是 `array[0][3] of int`。

你现在应该检查这几处

`type_checker.hpp`

确保是：

`</> C++`

```
void error(int code, AST::NodePtr node, const std::string& message);

TypePtr build_var_type(BasicType btype, const std::vector<int>& dims);
TypePtr build_param_type(AST::FuncParamPtr param);
```



`type_checker.cpp`

确保是：

`</> C++`



```
void TypeChecker::error(int code, AST::NodePtr node,
                        const std::string& message) {
```



```
void TypeChecker::Error(const Code, const Model & node,  
                        const std::string& message) {  
    ...  
}
```

以及：

```
</> C++  
TypePtr TypeChecker::build_var_type(BasicType btype,  
                                     const std::vector<int>& dims) {  
    ...  
}
```

和：

```
</> C++  
TypePtr TypeChecker::build_param_type(AST::FuncParamPtr param) {  
    ...  
}
```

一句话总结

这两个错误都是因为：

```
.hpp 里的函数声明  
和  
.cpp 里的函数实现  
参数列表对不上
```

把 `.hpp` 和 `.cpp` 的函数名、参数类型、`&` 都保持完全一致，就可以解决。

📄 🌟 🗨 📁 🔄 ...



```
if (array_type->dims.size() <= 1) {
    current_type = array_type->element_type;
} else {
    std::vector<int> remain_dims(array_type-
>dims.begin() + 1,
                                array_type->dims.end());
    current_type = ArrayType::create(array_type-
>element_type, remain_dims);
}这段代码是做什么用的
```

Thought for a couple of seconds >

这段代码是在处理数组被下标访问之后，类型应该怎么变化。

比如：

```
</> C
int a[2][3];
```

a 本身的类型是：

array[2][3] of int

当你写：

```
</> C
a[0]
```

它不再是整个二维数组，而是第一行，所以类型变成：

array[3] of int

当你继续写：

`</> C`

```
a[0][1]
```

它才真正变成：

```
int
```

你这段代码就是在做这个“降维”操作：

`</> C++`

```
if (array_type->dims.size() <= 1) {  
    current_type = array_type->element_type;  
} else {  
    std::vector<int> remain_dims(array_type->dims.begin() + 1,  
                                array_type->dims.end());  
    current_type = ArrayType::create(array_type->element_type, remain_dims);  
}
```

情况一：数组只剩一维

比如：

`</> C`

```
int a[5];  
a[0]
```

一开始：

a 的类型 = array[5] of int



此时：

```
</> C++
```



```
array_type->dims.size() == 1
```

说明它只有一维。

访问一次下标之后：

```
</> C
```



```
a[0]
```

类型就应该变成元素类型：

```
int
```



所以这里写：

```
</> C++
```



```
current_type = array_type->element_type;
```

意思是：

如果当前数组只剩最后一维，那么用一个下标访问后，结果就是数组元素，也就是 `int`。

情况二：数组还有多维

比如：

```
</> C
```



```
int a[2][3];  
a[0]
```



一开始:

```
a 的类型 = array[2][3] of int
```

此时:

```
</> C++  
array_type->dims.size() == 2
```

访问一次下标后:

```
</> C  
a[0]
```

类型应该从:

```
array[2][3] of int
```

变成:

```
array[3] of int
```

所以这段代码:

```
</> C++  
std::vector<int> remain_dims(array_type->dims.begin() + 1,  
                             array_type->dims.end());
```

意思是: 去掉第一维, 保留后面的维度。

例如:



```
</> C++  
dims = {2, 3}
```


 C++



执行后：

 C++



```
remain_dims = {3}
```

然后重新构造一个数组类型：

 C++



```
current_type = ArrayType::create(array_type->element_type, remain_dims);
```

也就是：

array[3] of int



再看三维例子：

 C



```
int a[2][3][4];
```

一开始：

a = array[2][3][4] of int



访问：

 C



```
a[0]
```



执行一次这段逻辑后：

```
array[3][4] of int
```



再访问：

```
</> C  
a[0][1]
```



再执行一次后：

```
array[4] of int
```



再访问：

```
</> C  
a[0][1][2]
```



最后变成：

```
int
```



一句话总结

这段代码的作用是：

每使用一个数组下标，就把数组类型降低一维；
如果已经是一维数组，再下标访问后就变成元素类型 `int`。



也就是：

```
int a[2][3][4];  
  
a      -> array[2][3][4] of int  
a[0]   -> array[3][4] of int
```

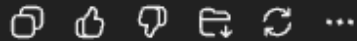


也就是：

```
int a[2][3][4];
```



```
a          -> array[2][3][4] of int  
a[0]       -> array[3][4] of int  
a[0][1]    -> array[4] of int  
a[0][1][2] -> int
```



`node->args.size()`这个字段以及方法的定义在哪里

Thought for 6s >

`node->args.size()` 里其实有两部分：

```
</> C++
```



```
node->args
```

和：

```
</> C++
```



```
.size()
```

它们的来源不一样。

1. `args` 字段定义在哪里？

`args` 是你们 AST 里 `FuncCall` 节点的成员变量，应该定义在：



大概是这个类：

</> C++



```
class FuncCall : public Node {
public:
    std::string name;
    std::vector<NodePtr> args;
    SymbolPtr symbol;

    FuncCall(std::string name) : name(std::move(name)) {}

    void add_arg(NodePtr exp) {
        args.push_back(exp);
    }

    std::string to_string() override {
        return "FuncCall <name: " + name + ">";
    }

    std::vector<NodePtr> get_children() override {
        return args;
    }
};
```

其中这一行：

</> C++



```
std::vector<NodePtr> args;
```

就是 `args` 字段的定义。

它表示：函数调用时传入的实参列表。

比如：



</> C



```
f(1, x + 2, y)
```

对应：

```
node->args = [1, x + 2, y]
```



所以：

```
</> C++
```



```
node->args.size()
```

就是这个函数调用传了几个参数。

2. `size()` 方法定义在哪里？

`size()` 不是你自己写的函数，而是 C++ 标准库 `std::vector` 自带的方法。

因为 `args` 的类型是：

```
</> C++
```



```
std::vector<NodePtr>
```

所以它天然有：

```
</> C++
```



```
args.size()
```

表示 `vector` 里面有多少个元素。

例如：

```
</> C++
```



```
std::vector<int> v = {1, 2, 3};  
std::cout << v.size(); // 输出 3
```

对应到你的代码：

```
</> C++
```

```
node->args.size()
```

表示：

当前函数调用有多少个实参

3. `node->args.size()` 整体是什么意思？

比如：

```
</> C
```

```
f(1, 2);
```

AST 中：

```
node->name = "f"  
node->args = [1, 2]
```

所以：

```
</> C++
```

```
node->args.size()
```

结果是：

2



在 `checkFuncCall` 里这句：

 C++ 

```
if (node->args.size() != func_type->param_types.size()) {  
    error(13, node, "function arguments number not matched");  
}
```

意思就是：

函数调用时传入的实参数量
是否等于
函数定义时声明的形参数量 

例如：

 C 

```
int f(int x) {  
    return x;  
}  
  
int main() {  
    f(1, 2);  
}
```

这里：

node->args.size() = 2
func_type->param_types.size() = 1 

数量不一样，所以报：

13 ArityMismatch 

4. 怎么在你项目里确认？



你可以运行：

 C



```
}
```

这里:

```
node->args.size() = 2  
func_type->param_types.size() = 1
```



数量不一样, 所以报:

```
13 ArityMismatch
```



4. 怎么在你项目里确认?

你可以运行:

 Bash



```
grep -n "class FuncCall" -A30 src/ast/tree.hpp
```

或者:

 Bash



```
grep -n "args" src/ast/tree.hpp src/parser/parser.y
```

你应该能看到:

 C++



```
std::vector<NodePtr> args;
```

以及 parser 里如何把函数调用参数放进 `ar` 

     ...


```
(base) x 🐍 base yangsmac@YangsdeMacBook-Air ~/Desktop/compiler/cp-3230104703 lab2 ±  
ls  
appends  compiler  lexer.c  parser.c  sp26-tests  
venv  
build.sh  config.toml  Makefile  reports  src  
(base) 🐍 base yangsmac@YangsdeMacBook-Air ~/Desktop/compiler/cp-3230104703 lab2 ±  
source .venv/bin/activate  
source: no such file or directory: .venv/bin/activate这是什  
么原因造成的
```

原因很简单：你当前目录里没有 `.venv` 这个文件夹。

你 `ls` 的结果里显示的是：

```
</> Bash
```

```
venv
```

不是：

```
</> Bash
```

```
.venv
```

所以你执行：

```
</> Bash
```

```
source .venv/bin/activate
```

系统找不到这个路径，就报：

```
source: no such file or directory: .venv/bin/activate
```



正确命令应该是

你现在应该执行：

```
</> Bash
```



```
source venv/bin/activate
```

注意前面没有点。

怎么确认？

你可以运行：

```
</> Bash
```



```
ls venv/bin/activate
```

如果存在，会输出：

```
venv/bin/activate
```



然后再执行：

```
</> Bash
```



```
source venv/bin/activate
```

成功后终端前面可能会变成：



```
</> Bash
```



如果存在，会输出：

```
venv/bin/activate
```



然后再执行：

```
</> Bash
```



```
source venv/bin/activate
```

成功后终端前面可能会变成：

```
</> Bash
```



```
(venv) ...
```

如果 `venv/bin/activate` 也不存在

那说明这个 `venv` 可能不是完整虚拟环境。可以检查：

```
</> Bash
```



```
ls venv
```

如果里面没有 `bin/activate`，那就重新创建：

```
</> Bash
```



```
python3 -m venv venv  
source venv/bin/activate
```

不过从你的 `ls` 看，当前问题只是路径写错了

